

LegalEase

Live Integration

Detailed Project Topic Document

Purpose: Connect the frontend, FastAPI backend and Gemini model so a real request travels end-to-end through the application.

The documentation defines Gemini as the AI integration point: the generator forms a structured prompt, calls the model, receives generated text, and returns it to FastAPI for display/editing/export.

■filecite■turn0file0■L137-L141■

End-to-end flow

- User enters document requirements in Streamlit
- Streamlit creates JSON and POSTs to /generate
- FastAPI validates the JSON with Pydantic
- GeminiDocumentGenerator constructs the prompt
- Gemini returns the draft
- FastAPI returns document_type and content
- Streamlit places content into the editable editor
- Edited content is sent to export endpoints

Gemini generator

```
from google import genai
from google.genai import types

class GeminiDocumentGenerator:
    def __init__(self, api_key: str, model_name: str):
        self.client = genai.Client(api_key=api_key)
        self.model_name = model_name

    def generate_document(self, document_type, parties, terms,
                          effective_date, jurisdiction="", additional_instructions=""):
        terms_block = "\n".join(f"- {t}" for t in terms)

        prompt = f'''
Draft a {document_type}.
Parties: {parties}
Effective date: {effective_date}
Jurisdiction: {jurisdiction or "[Not specified]"}
Terms:
{terms_block}
Additional instructions: {additional_instructions or "[None]"}

Do not invent missing facts. Use [INSERT ...] placeholders.
Return only the structured document draft.
'''

        result = self.client.models.generate_content(
            model=self.model_name,
            contents=prompt,
            config=types.GenerateContentConfig(
                temperature=0.2,
                max_output_tokens=12000,
            ),
        )
        return (result.text or "").strip()
```

Environment configuration

```
GEMINI_API_KEY=your_key_here
GEMINI_MODEL=gemini-3.8-flash
BACKEND_URL=http://127.0.0.1:8000
```

The original documentation identifies Gemini 1.5 Pro, but the implementation uses a configurable model and the current Google GenAI SDK rather than hard-coding the legacy SDK integration. The selected model can be changed in .env without changing application code.

Source basis: This topic is derived from the supplied LegalEase project documentation. Where the source did not provide implementation detail, this document presents the corresponding implementation structure used in the completed project.